

SIMATS ENGINEERING
ASSESSMENT TOOL 1-Pseudocode Writing Task

COURSE NAME: R PROGRAMMING

COURSE CODE: ITA04

COURSE OUTCOME COVERED

CO: Apply R programming fundamentals including data structures, vectors, matrices, arrays, and class types to perform mathematical operations and manage data effectively.

(Bloom Level -BL2- BL4)

Assessment Tool Weightage: 20%

QUESTIONS:

Total Marks: 100

1. Variables and Data Types Analysis (BL4 – Analyze)

Design pseudocode to simulate basic variable handling in R programming:

- Declare variables of different data types (numeric, character, logical)
- Perform arithmetic and logical operations
- Display the results

2. Data Structures Implementation (BL4 – Analyze)

Write pseudocode to demonstrate advanced data structures in R:

- Create and initialize a vector, list, and matrix
- Perform operations such as indexing, updating, and aggregation
- Compare the structures based on usage

3. Matrix and Array Operations (BL4 – Analyze)

Design pseudocode to perform operations on matrices and arrays:

- Create a matrix and a 3D array
- Perform addition, multiplication (matrix), and element-wise operations
- Display the results and analyze differences

4. Data Frame Construction and Manipulation (BL4 – Analyze))

A dataset needs to be organized efficiently.

Write pseudocode to:

- Create a data frame with multiple columns (numeric and character)
- Add, delete, and modify columns
- Perform basic summary operations

5. Functions and Logical Operations (BL4 – Analyze)

Design pseudocode to implement functions and logical vector operations:

- Create a user-defined function with input parameters
- Perform operations on logical vectors
- Return and display computed results

Code of Conduct:

I Adhithyan S (192421102) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

1.Student Marks Analysis (Vectors)

```
marks <- c(78, 65, 89, 91, 56, 72, 84, 95, 67, 80)
```

```
highest <- marks[1]
```

```
for(i in 2:length(marks)){
```

```
  if(marks[i] > highest)
```

```
    highest <- marks[i]
```

```
}
```

```
lowest <- marks[1]
```

```
for(i in 2:length(marks)){
```

```
  if(marks[i] < lowest)
```

```
    lowest <- marks[i]
```

```
}
```

```
sum_marks <- 0
```

```
for(i in 1:length(marks)){
```

```
  sum_marks <- sum_marks + marks[i]
```

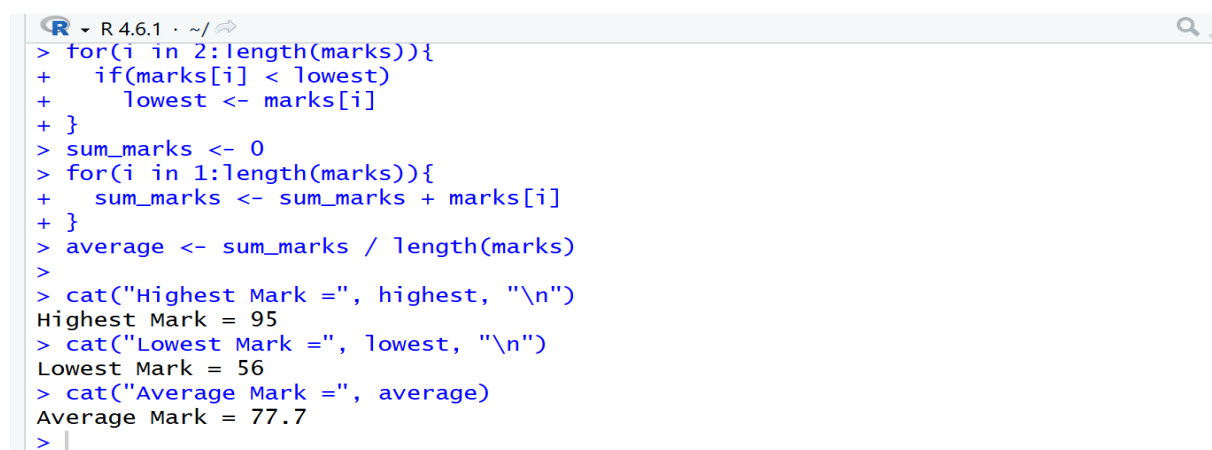
```
}
```

```
average <- sum_marks / length(marks)
```

```
cat("Highest Mark =", highest, "\n")
```

```
cat("Lowest Mark =", lowest, "\n")
```

```
cat("Average Mark =", average)
```



```
R 4.6.1 · ~/
> for(i in 2:length(marks)){
+   if(marks[i] < lowest)
+     lowest <- marks[i]
+ }
> sum_marks <- 0
> for(i in 1:length(marks)){
+   sum_marks <- sum_marks + marks[i]
+ }
> average <- sum_marks / length(marks)
>
> cat("Highest Mark =", highest, "\n")
Highest Mark = 95
> cat("Lowest Mark =", lowest, "\n")
Lowest Mark = 56
> cat("Average Mark =", average)
Average Mark = 77.7
> |
```

2. Employee Salary Matrix

```
salary <- matrix(c(
```

```

30000,5000,3000,
35000,6000,4000,
32000,5500,3500,
40000,7000,4500),
nrow=4, byrow=TRUE)
total <- salary[,1] + salary[,2] + salary[,3]
cat("Department-wise Total Salary\n")
print(total)
max_dept <- which.max(total)
cat("Department with Maximum Total Salary =", max_dept)

```

```

> salary <- matrix(c(
+   30000,5000,3000,
+   35000,6000,4000,
+   32000,5500,3500,
+   40000,7000,4500),
+   nrow=4, byrow=TRUE)
> total <- salary[,1] + salary[,2] + salary[,3]
> cat("Department-wise Total Salary\n")
Department-wise Total Salary
> print(total)
[1] 38000 45000 41000 51500
> max_dept <- which.max(total)
> cat("Department with Maximum Total Salary =", max_dept)
Department with Maximum Total Salary = 4
>

```

3. Rainfall Data (Arrays)

```

rainfall <- array(c(
120,140,130,150,
100,110,105,115,
90,95,100,110,
125,145,135,155,
105,115,110,120,
95,100,105,115),
dim=c(3,4,2))
annual <- c(0,0,0)
for(city in 1:3){

```

```

total <- 0

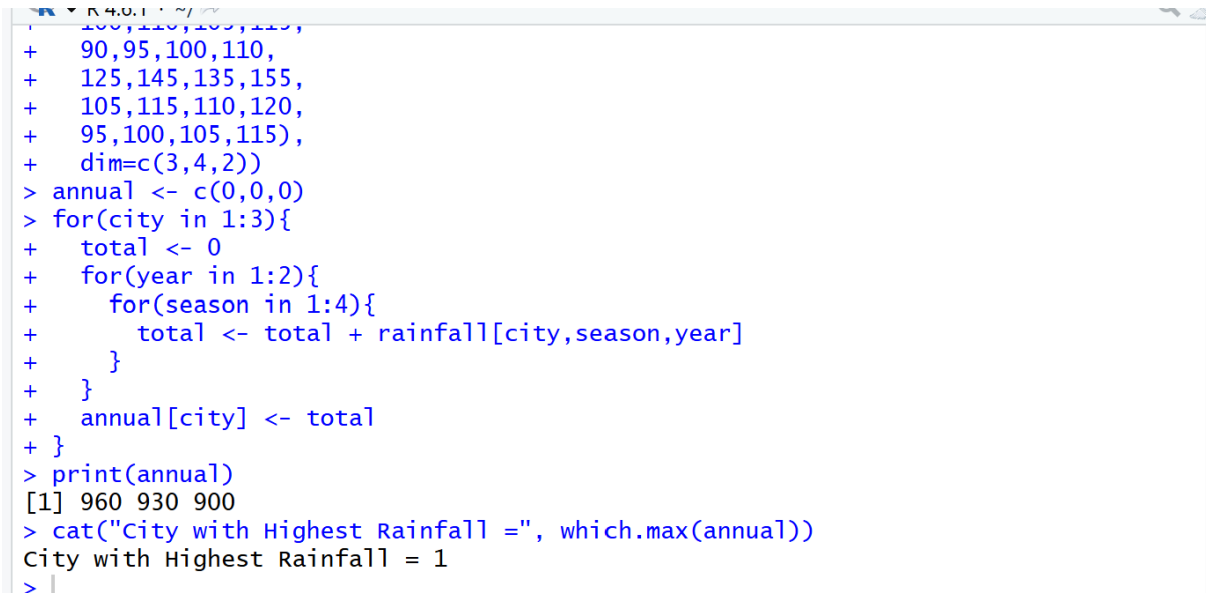
for(year in 1:2){
  for(season in 1:4){
    total <- total + rainfall[city,season,year]
  }
}

annual[city] <- total
}

print(annual)

cat("City with Highest Rainfall =", which.max(annual))

```



```

R 4.0.1 ~ / ~
+ 100,110,100,110,
+ 90,95,100,110,
+ 125,145,135,155,
+ 105,115,110,120,
+ 95,100,105,115),
+ dim=c(3,4,2))
> annual <- c(0,0,0)
> for(city in 1:3){
+   total <- 0
+   for(year in 1:2){
+     for(season in 1:4){
+       total <- total + rainfall[city,season,year]
+     }
+   }
+   annual[city] <- total
+ }
> print(annual)
[1] 960 930 900
> cat("City with Highest Rainfall =", which.max(annual))
City with Highest Rainfall = 1
> |

```

4. Hospital Patient Records (Lists)

```

patient <- list(
  ID=c(101,102,103,104),
  Name=c("Arun","Bala","Chitra","Deepa"),
  Age=c(45,68,72,58),
  BloodGroup=c("A+","B+","O+","AB+"),
  Bill=c(12000,18000,22000,15000)
)

```

```

cat("Patients above Age 60\n")
for(i in 1:length(patient$Age)){
  if(patient$Age[i] > 60){
    cat(patient$Name[i], "-", patient$Age[i], "\n")
  }
}

avg_bill <- sum(patient$Bill)/length(patient$Bill)
cat("Average Hospital Bill =", avg_bill)

```

```

> patient <- list(
+   ID=c(101,102,103,104),
+   Name=c("Arun","Bala","Chitra","Deepa"),
+   Age=c(45,68,72,58),
+   BloodGroup=c("A+","B+","O+","AB+"),
+   Bill=c(12000,18000,22000,15000)
+ )
> cat("Patients above Age 60\n")
Patients above Age 60
> for(i in 1:length(patient$Age)){
+   if(patient$Age[i] > 60){
+     cat(patient$Name[i], "-", patient$Age[i], "\n")
+   }
+ }
Bala - 68
Chitra - 72
> avg_bill <- sum(patient$Bill)/length(patient$Bill)
> cat("Average Hospital Bill =", avg_bill)
Average Hospital Bill = 16750
> |

```

5. Product Inventory (Factors)

```

products <- factor(c(
  "Electronics","Grocery","Stationery",
  "Grocery","Electronics","Stationery",
  "Grocery","Electronics","Grocery"))
levels(products) <- sort(levels(products))
print(levels(products))
count <- integer(length(levels(products)))
for(i in 1:length(products)){
  count[products[i]] <- count[products[i]] + 1
}

```

```

}

for(i in 1:length(levels(products))) {

  cat(levels(products)[i], "=", count[i], "\n")

}

> products <- factor(c(
+   "Electronics", "Grocery", "Stationery",
+   "Grocery", "Electronics", "Stationery",
+   "Grocery", "Electronics", "Grocery"))
> levels(products) <- sort(levels(products))
> print(levels(products))
[1] "Electronics" "Grocery"      "Stationery"
> count <- integer(length(levels(products)))
> for(i in 1:length(products)) {
+   count[products[i]] <- count[products[i]] + 1
+ }
> for(i in 1:length(levels(products))) {
+   cat(levels(products)[i], "=", count[i], "\n")
+ }
Electronics = 3
Grocery = 4
Stationery = 2
> |

```

6. Cricket Performance Analysis (Data Frames)

```

players <- data.frame(
  Name=c("Virat","Rohit","Gill","Rahul","Pant"),
  Matches=c(100,95,70,80,75),
  Runs=c(5000,4500,3200,3500,3400),
  Wickets=c(4,8,2,1,0),
  StrikeRate=c(130,118,125,115,135)
)

cat("Players with Strike Rate >120\n")

print(players[players$StrikeRate>120,])

avg_runs <- sum(players$Runs)/nrow(players)

cat("Average Runs =", avg_runs)

```

```

> players <- data.frame(
+   Name=c("Virat","Rohit","Gill","Rahul","Pant"),
+   Matches=c(100,95,70,80,75),
+   Runs=c(5000,4500,3200,3500,3400),
+   Wickets=c(4,8,2,1,0),
+   StrikeRate=c(130,118,125,115,135)
+ )
> cat("Players with Strike Rate >120\n")
Players with Strike Rate >120
> print(players[players$StrikeRate>120,])
  Name Matches Runs Wickets StrikeRate
1 Virat     100 5000        4        130
3  Gill      70 3200         2        125
5  Pant      75 3400         0        135
> avg_runs <- sum(players$Runs)/nrow(players)
> cat("Average Runs =", avg_runs)
Average Runs = 3920
> |

```

7. Bank Transaction Processing (Class Types)

```

bank <- data.frame(
AccountNo=c(1001,1002,1003),
Customer=c("Arun","Bala","Chitra"),
Balance=c(25000,32000,45000),
Status=c("Active","Inactive","Active")
)
cat("Class of Every Column\n")
print(sapply(bank,class))
bank$Status <- factor(bank$Status)
cat("After Conversion\n")
print(sapply(bank,class))
cat("Balance Column Class =", class(bank$Balance))

```

```

> bank <- data.frame(
+   AccountNo=c(1001,1002,1003),
+   Customer=c("Arun","Bala","Chitra"),
+   Balance=c(25000,32000,45000),
+   Status=c("Active","Inactive","Active")
+ )
> cat("Class of Every Column\n")
Class of Every Column
> print(sapply(bank,class))
AccountNo Customer Balance Status
"numeric" "character" "numeric" "character"
> bank$Status <- factor(bank$Status)
> cat("After Conversion\n")
After Conversion
> print(sapply(bank,class))
AccountNo Customer Balance Status
"numeric" "character" "numeric" "factor"
> cat("Balance Column Class =", class(bank$Balance))
Balance Column Class = numeric
> |

```

8. University Attendance Management (Vectors)

```
attendance <- c(95,82,88,91,75,86,79,93,84,90,87,80,92,89,81)
```

```
eligible <- attendance >= 85
```

```
ineligible <- attendance < 85
```

```
cat("Eligible Students\n")
```

```
print(attendance[eligible])
```

```
cat("Ineligible Students\n")
```

```
print(attendance[ineligible])
```

```
percentage <- (sum(eligible)/length(attendance))*100
```

```
cat("Eligible Percentage =", percentage,"%")
```

```

> attendance <- c(95,82,88,91,75,86,79,93,84,90,87,80,92,89,81)
> eligible <- attendance >= 85
> ineligible <- attendance < 85
> cat("Eligible Students\n")
Eligible Students
> print(attendance[eligible])
[1] 95 88 91 86 93 90 87 92 89
> cat("Ineligible Students\n")
Ineligible Students
> print(attendance[ineligible])
[1] 82 75 79 84 80 81
> percentage <- (sum(eligible)/length(attendance))*100
> cat("Eligible Percentage =", percentage,"%")
Eligible Percentage = 60 %
> |

```

9. Electricity Consumption Analysis (Matrices)

```

consumption <- matrix(c(
210,220,230,225,240,235,245,250,260,255,265,270,
180,185,190,195,200,205,210,215,220,225,230,235,
250,255,260,265,270,275,280,285,290,295,300,305,
160,165,170,175,180,185,190,195,200,205,210,215,
300,310,320,315,325,330,340,345,350,355,360,365),
nrow=5, byrow=TRUE)
annual <- rowSums(consumption)
print(annual)
cat("Building with Highest Consumption =", which.max(annual))

```

```

> consumption <- matrix(c(
+ 210,220,230,225,240,235,245,250,260,255,265,270,
+ 180,185,190,195,200,205,210,215,220,225,230,235,
+ 250,255,260,265,270,275,280,285,290,295,300,305,
+ 160,165,170,175,180,185,190,195,200,205,210,215,
+ 300,310,320,315,325,330,340,345,350,355,360,365),
+ nrow=5, byrow=TRUE)
> annual <- rowSums(consumption)
> print(annual)
[1] 2905 2490 3330 2250 4015
> cat("Building with Highest Consumption =", which.max(annual))
Building with Highest Consumption = 5
> |

```

10. Sales Performance Dashboard (Vectors, Matrices, Arrays, and Class Types)

```

monthly <- c(120,140,135,150,160,155,170,175,180,190,200,210)
quarterly <- matrix(c(
395,465,570,
410,480,600),
nrow=2, byrow=TRUE)
regional <- array(c(
120,130,
140,150,
160,170,

```

```
180,190,  
125,135,  
145,155,  
165,175,  
185,195),  
dim=c(2,4,2))
```

```
category <- factor(c(  
"Electronics","Grocery",  
"Stationery","Electronics"))
```

```
cat("Class Types\n")  
print(class(monthly))  
print(class(quarterly))  
print(class(regional))  
print(class(category))  
annual_sales <- sum(monthly)  
cat("Annual Sales =", annual_sales, "\n")  
region_total <- c(0,0)  
for(region in 1:2){  
  total <- 0  
  for(q in 1:4){  
    for(year in 1:2){  
      total <- total + regional[region,q,year]  
    }  
  }  
  region_total[region] <- total  
}  
cat("Best Performing Region =", which.max(region_total), "\n")
```

```
quarter_total <- colSums(quarterly)
```

```
cat("Quarterly Totals\n")
```

```
print(quarter_total)
```

```
> cat("Class Types\n")
Class Types
> print(class(monthly))
[1] "numeric"
> print(class(quarterly))
[1] "matrix" "array"
> print(class(regional))
[1] "array"
> print(class(category))
[1] "factor"
> annual_sales <- sum(monthly)
> cat("Annual Sales =", annual_sales, "\n")
Annual Sales = 1985
> region_total <- c(0,0)
> for(region in 1:2){
+   total <- 0
+   for(q in 1:4){
+     for(year in 1:2){
+       total <- total + regional[region,q,year]
+     }
+   }
+   region_total[region] <- total
+ }
> cat("Best Performing Region =", which.max(region_total), "\n")
Best Performing Region = 2
> quarter_total <- colSums(quarterly)
> cat("Quarterly Totals\n")
Quarterly Totals
> print(quarter_total)
[1] 805 945 1170
> |
```

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient